

Threshold Secure Certificateless Subgroup Multi-Signature for Anonymously Consensus Mechanisms

Zhiwei Wang  and Siuming Yiu

Abstract—Compared with threshold signatures, multi-signatures do not require the distributed key generation (DKG) protocol. However, the identities of participating signers must be public during verification. In this paper, we propose a certificateless subgroup multi-signature scheme with anonymity for consensus mechanisms. The certificateless construction eliminates public key management and resists key escrow. Our scheme employs Bulletproofs-based inner product arguments to aggregate public keys, achieving anonymity for each participating signer. Their public keys and identities are not disclosed during verification. Unlike threshold signatures, our scheme avoids the computational overhead of DKG, which is particularly significant in a certificateless setting. Any arbitrary subgroup can produce a valid multi-signature as long as its size exceeds a given threshold. We utilize the Univariate SumCheck lemma to enable threshold comparison without revealing subgroup members' identities. Thus, our scheme ensures both security and privacy for consensus mechanisms.

Index Terms—Certificateless, multi-signature, inner product argument, anonymous, threshold.

I. INTRODUCTION

IN PRACTICAL Internet of Things (IoT) and blockchain applications, a consensus mechanism often requires multiple users and minors to jointly sign a common message to establish its validity. A cryptographic primitive known as multi-signature is particularly well-suited for such scenarios. It enables a group of signers, each possessing their own secret/public key pair $\{(sk_i, pk_i)\}$, to collectively generate a single signature σ on a message m . This signature σ can be verified using the set of public keys $\{pk_1, \dots, pk_n\}$ to convince the verifier that every signer has approved m [1]. However, traditional multi-signature schemes [2], [3], [4], [5], [6] are predominantly implemented within the Public Key Infrastructure (PKI) framework. In this framework, each signer's public key must be certified by a Certificate Authority (CA). When a verifier receives a multi-signature, they can verify it using the public keys of all signers. It

is crucial to note that these public keys must also be authenticated to prevent public key substitution attacks. The process of issuing, transmitting, and verifying certificates introduces additional overhead and costs.

To address this issue, Shamir introduced the notion of identity-based cryptography, which uses publicly verifiable information such as names, addresses, and identity numbers as public keys [7]. Since then, numerous identity-based multi-signature schemes have been proposed [8], [9]. However, identity-based cryptosystems face an inherent challenge known as the key escrow problem. Specifically, to generate identity-based private keys for each user, a trusted key generation center (KGC) is required to participate in the cryptosystem, holding the system's primary secret key. It is evident that a malicious KGC could potentially deduce the leakage of every user's private keys. Thus, the trustworthiness of the KGC is crucial for ensuring the security of the entire system.

To address the challenges of certificate management and key escrow, Al-Riyami and Paterson introduced a pioneering cryptographic primitive known as the certificateless cryptosystem [10]. This innovation has garnered significant attention from the cryptographic research community, leading to the development of numerous certificateless cryptographic schemes. The key feature of certificateless cryptosystems is that a user's full secret key is composed of two parts: a partial identity-based private key generated by a KGC and a partial secret key chosen by the user themselves. This design ensures that the KGC has control only over the partial identity-based private keys, thereby preventing any malicious attempts by the KGC to impersonate users without knowledge of their partial secret keys. Despite these notable advantages, certificateless cryptosystems have faced several security concerns. Huang et al. identified potential public key attacks against these systems [12], while Au et al. demonstrated that a malicious KGC could potentially replace other users' secret/public key pairs [13]. These findings highlight the ongoing need for robust security mechanisms to mitigate such vulnerabilities in certificateless cryptographic schemes.

Furthermore, multi-signature schemes must be carefully designed to prevent rogue-key attacks [3]. In these attacks, an adversary can arbitrarily choose its public key and compute it as a function of valid users' public keys, thereby enabling the generation of forged multi-signatures with relative ease. Many early multi-signature schemes have been compromised by such rogue-key attacks [14], [15]. In PKI-based multi-signature

Received 7 December 2025; revised 16 July 2026; accepted 17 July 2026. Date of publication 20 July 2026; date of current version 1 September 2026. This work was partially supported by the National Natural Science Foundation of China under Grant 62372245. (Corresponding author: Zhiwei Wang.)

Zhiwei Wang is with the School of Computer Sciences, Nanjing University of Posts and Telecommunications, Nanjing 210023, China (e-mail: zhwang@njupt.edu.cn).

Siuming Yiu is with the Department of Computer Sciences, University of Hong Kong, Hong Kong.

Digital Object Identifier 10.1109/TDSC.2026.3715447

schemes, resistance to rogue-key attacks can be achieved if the CA requires users to provide proofs of knowledge of their secret keys during the registration process [16], [17]. However, in certificateless cryptosystems, there is no CA or other trusted central authority to enforce such requirements. As a result, it is highly desirable to construct certificateless multi-signature schemes that can withstand rogue-key attacks without relying on registration with a CA or any other trusted central party. This challenge underscores the need for innovative cryptographic techniques that can ensure the integrity and security of multi-signatures in the certificateless setting.

Another significant limitation of early multi-signature schemes is their inability to enable arbitrary subgroups to generate multi-signatures on behalf of the entire group when Byzantine users are present [18]. This shortcoming renders them unsuitable for consensus protocols in Byzantine agreement settings. Early attempts to address this issue include the work of Alexandra et al. who proposed the first subgroup multi-signature scheme. However, their definition and security model remained incomplete [19]. Subsequently, Elrond et al. introduced a subgroup multi-signature scheme based on the BLS signature in 2019, but it lacked a formal security proof [20]. More recently, Galindo et al. developed a provably secure subgroup multi-signature scheme derived from the BLS signature. While this scheme offered enhanced security guarantees, it still relied on the PKI system, which introduces additional complexities and dependencies [3].

Threshold signatures are a crucial cryptographic primitive employed in the consensus protocols of Byzantine agreement. They enable the distribution of a secret key among a group of signers, allowing each signer to generate a partial signature over a common message using their respective key share [19]. An aggregator can then combine these partial signatures into a single, unified signature once a sufficient number (exceeding a predefined threshold) is received. Compared to multi-signatures, threshold signatures offer enhanced privacy protection. They do not disclose the identities of participating signers, as they do not require the public keys of the signers. This makes threshold signatures particularly suitable for privacy-preserving scenarios. However, a significant drawback of threshold signatures lies in the distributed key generation (DKG) protocol [21], [22], [23], [26]. First, the DKG protocol typically requires a trusted third party to execute it. Second, the protocol must be re-executed whenever the threshold value is altered. In certificateless cryptosystems, the situation is even more complex. When the threshold changes, the DKG protocol must be executed twice, as two separate secret keys (the partial identity-based private key and the partial secret key) need to be distributed. Consequently, frequent changes to the threshold result in prohibitively high costs associated with executing the DKG protocol. Given these challenges, the design of an anonymous certificateless subgroup multi-signature scheme becomes highly significant. Such a scheme would need to balance the privacy and flexibility required in dynamic environments while mitigating the complexities and costs associated with frequent threshold changes and distributed key generation.

If the verification of a certificateless multi-signature could be performed without requiring the public keys and identities of the participating signers, then only the aggregated public key would need to be revealed in the verification algorithm. However, enabling an aggregator to provide a zero-knowledge proof to the verifier for the aggregated public key, while concealing individual public keys and identities, poses a significant challenge. To address this, we represent the public keys, identities, and hash values of participating signers as vectors. The aggregated public key can then be expressed as the inner product of these vectors. This approach allows for the aggregation of public keys and identities from arbitrary subgroups of signers, with absent signers simply set to zero in the vectors. Existing zero-knowledge protocols for vector-value inner product arguments can be broadly categorized into two types. The first category is succinct non-interactive arguments of knowledge (SNARKs), which typically require verifier time costs of $O(n)$ and produce a transcript size of $O(n)$, where n is the length of the vector [27], [28], [29]. The second category is interactive arguments of knowledge, which are significantly more efficient than SNARKs. Among the existing interactive zero-knowledge protocols [24], [30], [31], [32], [34], Bulletproofs stands out as the most efficient, achieving verifier time complexity of $O(\log n)$ and logarithmic transcript size [24]. Therefore, we opt for a Bulletproofs-based inner product argument to implement the anonymous and arbitrary subgroup functionality in our certificateless multi-signature scheme. Another critical challenge in this design is ensuring that the number of participating signers exceeds a predefined threshold, similar to threshold signatures, even though the verifier cannot know the identities of the participants. Otherwise, the anonymity of the multi-signature scheme would be compromised. To address this, we incorporate Sourav et al.'s sum-check method [25], which allows the verifier to confirm the required number of participants without revealing their identities.

In recent years, Accountable Subgroup Multi-signature (ASM) [11] has emerged as an enhanced multi-signature primitive designed to provide signer anonymity while permitting authorized entities to trace signer identities when necessary—thereby achieving “controlled anonymity”. However, compared with the certificateless subgroup multi-signature scheme based on Bulletproofs inner product argument proposed in this paper, ASM offers several disadvantages. 1) ASM usually depends on trusted third parties or full identity-based frameworks, which introduce single points of failure or key leakage risks. 2) Although ASM supports accountability, its anonymity often relies on complex group or ring signatures, resulting in higher verification overhead. 3) ASM and most threshold signature schemes require re-sharing key shares upon threshold changes, incurring significant costs in frequently changing settings.

Potential Application Scenarios of Our Scheme: 1) Anonymous Collective Decision-Making or Alerting in IoT Networks: Various IoT devices (e.g., environmental sensors, cameras) managed by different entities in an area need to reach a consensus on an event (e.g., pollution spike, security breach) and file a joint report, but device identity or ownership may be sensitive. The

subset of devices detecting the event generates an anonymous joint report signature. The receiving server can be confident the report originates from a sufficient number (exceeding a threshold) of legitimate devices without identifying the exact reporters, protecting operational privacy. The certificateless property eliminates the burden of certificate management for vast IoT fleets. 2) Privacy-Sensitive Transaction Authorization in Enterprise/Consortium Chains: A significant transaction requires approval from multiple executives or departments (a subgroup). For internal confidentiality, it may be undesirable to reveal which specific individuals approved a particular transaction. Approvers use the scheme to create an anonymous multi-signature on the transaction hash. The transaction record (on-chain or in a database) only stores this anonymous aggregated signature and a proof of the number of approvers. Auditors can verify that the transaction received sufficient authorization without knowing the approvers' identities.

Contributions: The main contributions of this paper are as follows: 1) We design an inner product argument for the aggregated public key in the certificateless subgroup multi-signature scheme, based on Bulletproofs. We provide rigorous proofs for the perfect completeness and computational knowledge soundness of this argument. This foundational component enables the aggregation of public keys while preserving the anonymity of participating signers. 2) We construct a certificateless subgroup multi-signature scheme that achieves signer anonymity while ensuring that the subgroup size exceeds a specified threshold. The anonymity property is achieved through the inner product argument for the aggregated public key, and the threshold comparison is implemented using the Univariate SumCheck lemma. We provide comprehensive proofs for the unforgeability of our scheme, thereby guaranteeing its security and robustness. 3) We conduct a thorough performance evaluation of our proposed scheme and compare it with other related schemes in the literature. This analysis highlights the efficiency and practicality of our approach, demonstrating its potential for real-world applications.

Organization: Some preliminaries, such as mathematical concepts, BLS signature, definition of zero-knowledge argument, symbol notations and Bulletproofs are introduced in Section II. Section III presents the designing of inner product argument of aggregated public key and the construction of our certificateless subgroup multi-signature which has the properties of anonymous and given threshold. In Section IV, we test our scheme's performances. We conclude our paper in Section V.

II. PRELIMINARY

A. Mathematical Concepts and BLS Signature

In this section, we review the related mathematical concepts for our scheme construction and security analysis.

We assume that $\mathbb{G}$ and $\mathbb{G}_T$ are two cyclic additive groups with the prime order q . We define $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ as the bilinear map that satisfies the following properties:

- 1) Bilinear: $\forall G_1, G_2 \in \mathbb{G}, \alpha_1, \alpha_2 \in \mathbb{Z}_q, e(\alpha_1 G_1, \alpha_2 G_2) = \alpha_1 \alpha_2 e(G_1, G_2)$.
- 2) Non-degenerate: $\exists G \in \mathbb{G}, e(G, G) \neq 1$.

- 3) Efficient Computability: There exists an efficient algorithm to compute $e(G_1, G_2)$ for all $G_1, G_2 \in \mathbb{G}$.

Next, we define the Computational Diffie-Hellman (CDH) Assumption [3] over $\mathbb{G}$ as follows.

CDH Assumption: Let $Gen(1^\iota)$ be a group generation algorithm, which takes a security parameter ι as input and outputs a description of a prime order group $\Theta = \{q, \mathbb{G}, \mathbb{G}_T, e\}$. The CDH Assumption over group $\mathbb{G}$ states that for any probabilistic polynomial-time (PPT) attackers, given a tuple $(G, \alpha G, \beta G)$ for a randomly chosen $\alpha, \beta \leftarrow_R \mathbb{Z}_q$ and $G \leftarrow_R \mathbb{G}$, the advantage for obtaining a solution $\alpha\beta G$ is negligible in ι .

Next, we review the BLS short signature [35] as follows.

BLS Signature: Let $(q, \mathbb{G}, \mathbb{G}_T, e, G)$ be a bilinear group with order q , and $G \in \mathbb{G}$. Let $H : \{0, 1\}^* \rightarrow \mathbb{G}$ be a cryptographic hash functions. Randomly chooses $s \in \mathbb{Z}_q^*$ as a secret key for a signer, and $pk = sG$ is its public key. The signing algorithm takes a message $m \in \{0, 1\}^*$ as inputs, and outputs $\sigma = s \cdot H(m)$ as the signature. The verifier checks whether $e(\sigma, G) = e(H(m, pk))$ holds, and outputs 1 if it holds, otherwise outputs 0.

B. Definition of Zero-Knowledge Argument

A zero-knowledge argument protocol allows a probabilistic polynomial-time (PPT) prover $\mathcal{P}$ to convince a verifier that $\mathcal{P}$ knows a witness ω for a statement s , without revealing any information about why s holds. We define a polynomial-time decidable relation $R \subset \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^*$. If $(SRS, s, \omega) \in R$, where SRS is a *structured reference string*, then ω is a valid witness for the statement s . The language dependent on SRS is defined as:

$$\mathcal{L}_{SRS} = \{s \mid \exists \omega \text{ such that } (SRS, s, \omega) \in R\},$$

which is the set of statements s that correspond to a witness ω in the relation R .

Generally, an argument of knowledge consists of three polynomial-time algorithms $(Setup, \mathcal{P}, \mathcal{V})$ [36]. The algorithm Setup outputs a structured reference string SRS on input the security parameter 1^λ . The prover $\mathcal{P}$ and verifier $\mathcal{V}$ interact on $(SRS, s : \omega)$ to produce a transcript T based on the verifier's randomness η . Here, $(SRS, s : \omega)$ represents the view of the prover, while (SRS, s, T, η) represents the view of the verifier. The verifier rejects if $\mathcal{V}(SRS, s, T, \eta) = \text{False}$; otherwise, if $\mathcal{V}(SRS, s, T, \eta) = \text{True}$, the verifier accepts.

If the triple $(Setup, \mathcal{P}, \mathcal{V})$ satisfies *perfect completeness* and *computational knowledge soundness*, then it is considered as an **argument of knowledge**.

Perfect Completeness: The triple $(Setup, \mathcal{P}, \mathcal{V})$ has the property of perfect completeness if for any PPT adversary $\mathcal{A}$:

$$\Pr \left(\begin{array}{c} (SRS, s : \omega) \notin R \vee \\ \mathcal{V}(SRS, s : T, \eta) = \text{True} \end{array} \middle| \begin{array}{c} SRS \leftarrow Setup(1^\lambda) \\ (s, \omega) \leftarrow \mathcal{A}(SRS) \\ T \leftarrow (\mathcal{P}, \mathcal{V})(SRS, s : \omega) \end{array} \right) = 1$$

Computational Knowledge Soundness: We say that a triple $(Setup, \mathcal{P}, \mathcal{V})$ for the relation R is *knowledge sound* with error $\kappa(|\vartheta|)$ if there exists a knowledge extractor algorithm $\mathcal{EK}$ with

the following properties. Given a statement s and a potentially dishonest prover $\mathcal{P}^*$ (which can be considered as a black-box oracle accessed by $\mathcal{EK}$), $\mathcal{EK}$ runs in polynomial time and outputs a witness $(\text{SRS}, s : \omega) \in R$ with probability:

$$\Pr[(\text{SRS}, s : \mathcal{EK}(s)) \in R] \geq \frac{p(\mathcal{P}^*, s) - \kappa(|\vartheta|)}{\text{poly}(|\vartheta|)},$$

where $p(\mathcal{P}^*, s)$ denotes the probability that the prover $\mathcal{P}^*$ generates an acceptable transcript.

C. Symbols and Notations

Groups and Fields: We begin by defining $\mathbb{G}$ as a cyclic group of prime order q , and Z_q as the corresponding finite field. The multiplicative group of nonzero elements in Z_q is denoted by $Z_q^* = Z_q \setminus \{0\}$. Group elements are represented by uppercase letters, such as $G, C, D, V, Y \in \mathbb{G}$, while field elements are represented by lowercase letters, such as $a, s, r, v, w, \tau, \mu, x \in Z_q$.

For operations, we use $+$ to denote addition in the field Z_q , and $\oplus$ to denote addition in the group $\mathbb{G}$. We use $\bigoplus_{i=\text{start}}^{\text{end}}$ to represent the summation of multiple group elements with index i ranging from start to end. Correspondingly, we use $\sum_{i=\text{start}}^{\text{end}}$ to represent the summation of multiple field elements with index $i \in [\text{start}, \text{end}]$.

Vector Operations: We use the bold letter $\mathbf{a}$ to represent a vector. We use the uppercase letter (e.g., $\mathbf{G}$) and the lowercase letter (e.g., $\mathbf{a}$) to distinguish a group vector (a vector of group elements) and a field vector (a vector of field elements). We use $\langle \cdot \rangle$ to denote the inner product operation between two vectors. Additionally, $\langle \mathbf{a}, \mathbf{G} \rangle$ denotes the inner product between a field vector $\mathbf{a}$ and a group vector $\mathbf{G}$.

We divide a vector $\mathbf{a} = (a_1, \dots, a_n)$ into the left half as $\mathbf{a}_L = (a_1, \dots, a_{n/2})$ and the right half as $\mathbf{a}_R = (a_{n/2+1}, \dots, a_n)$, and we use $(\mathbf{a}_L || \mathbf{a}_R) = \mathbf{a}$ to denote this procedure.

D. Bulletproofs

Bulletproofs is an efficient argument of knowledge with logarithmic proof size and logarithmic verification time. The communication costs of Bulletproofs are approximately $2 \log_2 n$ group elements and 2 field elements [24].

In Bulletproofs, the prover $\mathcal{P}$ aims to convince the verifier $\mathcal{V}$ that $\mathcal{P}$ knows two vectors $\mathbf{a}, \mathbf{b} \in Z_q^n$ such that $C = \langle \mathbf{a}, \mathbf{b} \rangle G$, where C is a commitment in $\mathbb{G}$ and G is an element in $\mathbb{G}$. Two vectors of group elements $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$ also should be provided in the conditions. Bulletproofs can be described as the following relation:

R_{bullet}

$$= \left\{ \left(\begin{array}{c} C, G \in \mathbb{G} \\ \mathbf{G}, \mathbf{H} \in \mathbb{G}^n \end{array} \right) : (\mathbf{a}, \mathbf{b} \in Z_q^n) \left| \left(\begin{array}{c} C = \langle \mathbf{a}, \mathbf{b} \rangle G \\ \oplus \langle \mathbf{a}, \mathbf{G} \rangle \\ \oplus \langle \mathbf{b}, \mathbf{H} \rangle \end{array} \right) \right. \right\}$$

We simplify Bulletproofs by eliminating the need for the vector $\mathbf{H}$ to hide $\mathbf{b}$. This is because, in our construction, $\mathbf{b}$ represents the vector of secret keys, which is already hidden within the vector of public keys $\mathbf{pk}$. $\mathcal{P}$ does not know $\mathbf{b}$, and the signers assist $\mathcal{P}$ in the inner product argument. The relation is described as follows, and the corresponding algorithm can be

Algorithm 1: Simple Version of Bulletproofs.

```

1: Input: Public input:  $C, G \in \mathbb{G}$ , and  $\mathbf{G} \in \mathbb{G}^n$ ;
2:  $\mathcal{P}$ 's private input:  $\mathbf{a}, \mathbf{b} \in Z_q^n$ .
3: Output: True or False ( $\mathcal{V}$  accepts or rejects)
4: if  $n = 1$  then
5:    $\mathcal{P} \rightarrow \mathcal{V} : a \in Z_q$ ;
6:    $\mathcal{V}$  checks whether  $C = abG \oplus aG$  holds,
7:   if it holds, returns True; otherwise, returns False.
8: else
9:    $\mathcal{P}$  and  $\mathcal{V}$  compute:  $(\mathbf{G}_l || \mathbf{G}_r) \leftarrow \mathbf{G}$  and  $n' \leftarrow n/2$ .
10:   $\mathcal{P}$  (with the help of others) computes:  $(\mathbf{a}_l || \mathbf{a}_r) \leftarrow \mathbf{a}$ ,
     $(\mathbf{b}_l || \mathbf{b}_r) \leftarrow \mathbf{b}$ ;
11:   $L = \langle \mathbf{a}_l, \mathbf{b}_r \rangle G \oplus \langle \mathbf{a}_l, \mathbf{G}_r \rangle \in \mathbb{G}$ ,
12:   $R = \langle \mathbf{a}_r, \mathbf{b}_l \rangle G \oplus \langle \mathbf{a}_r, \mathbf{G}_l \rangle \in \mathbb{G}$ ;
13:   $\mathcal{P} \rightarrow \mathcal{V} : L, R$ ;
14:   $\mathcal{V} \rightarrow \mathcal{P} : \gamma \leftarrow_R Z_q^* \triangleright \gamma$  is a challenge
15:   $\mathcal{P}$  computes:  $\mathbf{a}' \leftarrow \mathbf{a}_l + \gamma^{-1} \mathbf{a}_r \in Z_q^{n'}$ ,  $\mathbf{b}' \leftarrow$ 
     $\mathbf{b}_l + \gamma^{-1} \mathbf{b}_r \in Z_q^{n'}$ ;
16:   $\mathcal{P}$  and  $\mathcal{V}$  compute:  $C' = \gamma \cdot L \oplus C \oplus \gamma^{-1} R$ ;
17:   $\mathbf{G}' \leftarrow \mathbf{G}'_l \oplus \mathbf{G}'_r \in \mathbb{G}^{n'}$ ;
18:   $\mathcal{P}$  and  $\mathcal{V}$  continues to execute this algorithm on input
    as  $(C', G, \mathbf{G}' : \mathbf{a}', \mathbf{b}')$ 
19: end if

```

seen in Algorithm 1.

$R_{s\text{-bullet}}$

$$= \left\{ \left(\begin{array}{c} C, G \in \mathbb{G} \\ \mathbf{G} \in \mathbb{G}^n \end{array} \right) : (\mathbf{a}, \mathbf{b} \in Z_q^n) \left| \left(\begin{array}{c} C = \langle \mathbf{a}, \mathbf{b} \rangle G \\ \oplus \langle \mathbf{a}, \mathbf{G} \rangle \end{array} \right) \right. \right\}$$

To simplify the verification process, we can denote the entire verification as:

$$C \oplus \bigoplus_{j=1}^n (\gamma_j L_j \oplus \gamma_j^{-1} R_j) = \hat{a} \hat{b} G \oplus \hat{a} \langle \mathbf{t}, \mathbf{G} \rangle,$$

where γ_j is the challenge in the j th round, and $\hat{a}, \hat{b}$ are the field elements generated in the final round of the algorithm. $\mathbf{t} = (t_1, \dots, t_n)$ is a vector that is computed from the challenges as follows.

$$t_i = \prod_{j=1}^{\log_2 n} \gamma_j^{\theta_{i,j}},$$

where $\theta_{i,j}$ is the j th bit of $i-1$ according to the big-endian format. The above algorithm using Gentry et al.'s method which optimizes the entire verification process to a single multi-exponentiation. Obviously, this simple version of Bulletproofs scheme is an argument of knowledge if Bulletproofs is an argument of knowledge.

III. CERTIFICATELESS SUBGROUP MULTI-SIGNATURE FROM INNER PRODUCT ARGUMENT

A. Inner Product Argument of Aggregated Public Keys and Identities

In our construction, each signer selects its secret key randomly. Let $\mathbf{s} = (s_1, \dots, s_n) \in Z_p^n$ be the vector of secret keys, and the vector of public keys is $\mathbf{pk} = (s_1 G, \dots, s_n G) \in \mathbb{G}^n$.

Let $\mathcal{ID} = \{ID_1, \dots, ID_n\}$ be the set of identities and $\mathbf{ID} = (H_1(ID_1), \dots, H_1(ID_n)) \in \mathbb{G}^n$ be the vector of hashed identities where $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}$. In this section, we formulate the argument for aggregated public keys and identities as a relation R_{pkid} , which consists of two inner products. The first inner product is between the vector of public keys and a vector of random numbers indicating the signers who participates the signing, and the second inner product is between the vector of identities and the vector of random numbers. For a message m , the prover $\mathcal{P}$ sets a vector $\mathbf{a} = (a_1, \dots, a_n)$ where $a_i = 0$ if the i th signer does not participate the signing otherwise $a_i = H_2(ID_i, pk_i, \mathcal{ID}, \mathbf{pk})$ where $H_2 : \{0, 1\}^* \rightarrow Z_q$. The vector $\mathbf{a}$ is designed for resisting the rogue-key attacks. It computes a commitment c_a to the vector $\mathbf{a}$. The prover $\mathcal{P}$ secretly sets a bit vector $\mathbf{d} = (d_1, \dots, d_n)$ where $d_i = 0$ if $a_i = 0$ otherwise $d_i = 1$. The bit vector $\mathbf{d}$ can be used to count the subgroup size of participating signers by computing inner product $\langle \mathbf{d}, \mathbf{d} \rangle$.

As described above, R_{pkid} is an inner product argument between $\mathcal{P}$ and $\mathcal{V}$. The aggregated public key $G_{pk} = \langle \mathbf{a}, \mathbf{pk} \rangle$ and the aggregated identity $G_{id} = \langle \mathbf{a}, \mathbf{ID} \rangle$ are naturally inner products which are also Pederson vector commitments [36]. Let $\mathbf{a} = (a_1, \dots, a_n) \in Z_q^n$ be a secret vector for the anonymous of participating signers, and $\mathbf{G} = (G_1, \dots, G_n) \in \mathbb{G}^n$ be a public vector of group elements. A Pederson vector commitment can hide $\mathbf{a}$ as $\langle \mathbf{a}, \mathbf{G} \rangle = a_1 G_1 \oplus a_2 G_2 \oplus \dots \oplus a_n G_n$ where $\oplus$ denote the additive operation of group elements. Note that the inner product $\langle \mathbf{a}, \mathbf{pk} \rangle$ is essentially $\langle \mathbf{a}, \mathbf{s} \rangle G$. To keep up with the Bulletproofs, we want $\mathcal{P}$ to give inner product argument for $\langle \mathbf{a}, \mathbf{s} \rangle$ which is an inner product of two vectors of field elements.

Although the prover $\mathcal{P}$ does not know the vector of secret keys $\mathbf{s}$, the signers can assist $\mathcal{P}$ without additional online interaction. Specifically, during the individual signing phase, each signer precomputes and outputs not only their partial signature but also auxiliary values derived from their secret key s_i and the challenges that will be generated during the inner product argument. The aggregator $\mathcal{P}$ collects these precomputed values and can then execute the inner product argument locally, without requiring further communication with the signers. This approach preserves the standard multi-signature model where each signer participates only once. The auxiliary values are designed such that they do not leak additional information about the secret keys and can be securely combined by $\mathcal{P}$. We define the relation for the argument of aggregated public keys and identities R_{pkid} as follows:

$$R_{pkid} = \left\{ \left(\begin{array}{c} G_{pk}, G_{id}, D \in \mathbb{G}, \\ G \in \mathbb{G}, \\ \mathbf{ID} \in \mathbb{G}^n \end{array} \right) : \left(\begin{array}{c} \mathbf{a}, \mathbf{s} \\ \in Z_q^n, \\ \mathbf{d} \in Z_2^n \end{array} \right) \left| \begin{array}{c} G_{pk} = \langle \mathbf{a}, \mathbf{s} \rangle G \\ \wedge G_{id} = \langle \mathbf{a}, \mathbf{ID} \rangle \\ \wedge D = \langle \mathbf{a}, \mathbf{d} \rangle G \end{array} \right. \right\}$$

The corresponding algorithm can be seen in Algorithm 2. The algorithm follows a modular design: it first establishes a binding between the aggregated public key and the secret vector using

Algorithm 2: Aggregated Public Keys and Identities Argument.

- 1: **Input:** Public input: $G_{pk}, G_{id}, D, G \in \mathbb{G}$ and $\mathbf{ID} \in \mathbb{G}^n$;
 - 2: $\mathcal{P}$'s private input: $\mathbf{a}, \mathbf{s} \in Z_q^n, \mathbf{d} \in Z_2^n$, along with precomputed auxiliary values from signers.
 - 3: **Output:** *True* or *False* ($\mathcal{V}$ accepts or rejects)
 - 4: $\mathcal{P} \rightarrow \mathcal{V}$: $R \leftarrow rG, V \leftarrow vG$, and $r, v \leftarrow_R Z_q$;
 - 5: $\mathcal{V} \rightarrow \mathcal{P}$: $h, w \leftarrow_R Z_q^*$ $\triangleright h, w$ are two challenges
 - 6: $\mathcal{P} \rightarrow \mathcal{V}$: $z = r + h \langle \mathbf{a}, \mathbf{s} \rangle, u = v + w \langle \mathbf{a}, \mathbf{d} \rangle, x \leftarrow_R Z_q^*$ $\triangleright x$ is another challenge
 - 7: $\mathcal{V}$: returns *False* if $zG \neq R \oplus hG_{pk}$ or $uG \neq V \oplus wD$;
 - 8: $\mathcal{P}$ and $\mathcal{V}$ compute: $C = xG_{pk} \oplus G_{id}$;
 - 9: $\mathcal{P}$ and $\mathcal{V}$ execute the simple version of Bulletproofs on input as $(C, xG, \mathbf{ID} : \mathbf{a}, \mathbf{s})$, where $\mathcal{P}$ uses the precomputed auxiliary values to simulate the signers' responses to the Bulletproofs challenges.
-

a Schnorr-like proof (steps 4C7), then reduces the aggregated public key and identity to a single Pedersen commitment C , and finally invokes the simple Bulletproofs protocol (Algorithm 1) to prove knowledge of the witness vectors $\mathbf{a}, \mathbf{s}$ for the resulting commitment. This decomposition separates the zero-knowledge argument for the subgroup composition from the inner product proof, making the overall structure more modular and easier to analyze.

In the aggregated public keys and identities argument, the entire verification process can be denoted as the following form:

$$xG_{pk} \oplus G_{id} \oplus \oplus_{j=1}^n (\gamma_j L_j \oplus \gamma_j^{-1} R_j) = \hat{a} \hat{s} xG \oplus \hat{a} \langle \mathbf{t}, \mathbf{G} \rangle,$$

where $\hat{a}, \hat{s}$ are the field elements generated in the final round of the algorithm, $\mathbf{t}$ and other symbols are defined as above.

Anonymity Mechanism: The aggregated public key $G_{pk} = \langle \mathbf{a}, \mathbf{pk} \rangle$ and aggregated identity $G_{id} = \langle \mathbf{a}, \mathbf{ID} \rangle$ are inner products of the secret vector $\mathbf{a}$ with the public vectors. Since $\mathbf{a}$ is kept secret, the verifier cannot deduce from G_{pk} and G_{id} which indices i have non-zero a_i (i.e., who participated). Throughout the interactive argument (Algorithm 2 and the nested Algorithm 1), the prover and verifier exchange messages culminating in the verifier checking a series of group arithmetic equations (e.g., the final verification formula). The protocol never reveals the concrete values of the secret vectors $\mathbf{a}$ and $\mathbf{s}$. The verifier only learns that "there exists some valid subgroup of participants whose keys aggregate to G_{pk} and G_{id} according to the rules," but gains zero knowledge about the composition of that subgroup.

Resisting Rogue-Key Attacks: There are two primary methods to resist rogue-key attacks, which allow an adversary to undermine other signers' signing rights by using a crafted public key and identity. The first method is called *Proof-of-Processing* (PoP) [38]. For any claimed public key $pk = sG$, the signer computes the PoP as

$$\pi = sH_{\text{PoP}}(sG),$$

where $H_{\text{PoP}} : \{0, 1\}^* \rightarrow \mathbb{G}$ is a cryptographic hash function. The PoP verification passes if

$$e(G, \pi) = e(pk, H_{\text{PoP}}(sG)).$$

However, the cost of PoP generation and verification becomes prohibitively high when the number of public keys is large. The second method involves using hashed values derived from the related public keys and identities to combine individual signatures [3]. This approach is more efficient, especially when dealing with a large number of public keys. Considering efficiency, we choose the second method to resist rogue-key attacks.

By incorporating the hash value a_i as a scalar multiplier during the signature aggregation, our scheme binds each signer's contribution to the current complete set of signers. This prevents an adversary from pre-constructing a malicious public key in isolation that could manipulate the aggregate result, thereby effectively resisting rogue-key attacks.

Intuition: Our scheme achieves anonymity by hiding the selector vector $\mathbf{a}$ in Pedersen commitments G_{pk} and G_{id} . The verifier sees only aggregated values, not individual participants. The Bulletproofs protocol proves knowledge of $\mathbf{a}$ without revealing which entries are non-zero, thus preserving signer privacy while ensuring correctness.

B. Proof for Argument of Knowledge

Theorem 1. The aggregated public keys and identities argument is an argument of knowledge if the discrete logarithm assumption holds over the group $\mathbb{G}$ and the simple version of Bulletproofs is an argument of knowledge.

Proof. Perfect Completeness. We can prove that if the prover $\mathcal{P}$ have the valid witness for R_{pkid} , then it also have the valid witness of $R_{s-bullet}$ by using the following substitution.

$$G \leftarrow xG, \mathbf{G} \leftarrow \mathbf{ID},$$

$$C = \langle \mathbf{a}, \mathbf{b} \rangle G \oplus \langle \mathbf{a}, \mathbf{G} \rangle \leftarrow \langle \mathbf{a}, \mathbf{s} \rangle G \oplus \langle \mathbf{a}, \mathbf{ID} \rangle.$$

Since the perfect completeness of $R_{s-bullet}$ is obviously from Bulletproofs, then $\mathcal{P}$ can generate an acceptable transcript for the verifier of $R_{s-bullet}$. Thus, the verifier in Algorithm 2 line 9 returns *True*. Furthermore, the verification check in the line 7 of Algorithm 2 also returns *True*, since

$$zG = (r + h \langle \mathbf{a}, \mathbf{s} \rangle)G = rG \oplus h \langle \mathbf{a}, \mathbf{s} \rangle G = R \oplus hG_{pk},$$

$$uG = (v + w \langle \mathbf{a}, \mathbf{d} \rangle)G = vG \oplus w \langle \mathbf{a}, \mathbf{d} \rangle G = V \oplus wD.$$

Thus, $\mathcal{P}$ in Algorithm 2 can generate acceptable transcript for the aggregated public keys and identities argument.

Computational Knowledge Soundness. To prove the computational knowledge soundness, we firstly construct an extractor EK_{DL} for $G_{pk} = sG$ to output s if the discrete logarithm assumption holds. And secondly, we construct an efficient extractor EK_{semi} for $C = \langle \mathbf{a}, \mathbf{s} \rangle G \oplus \langle \mathbf{a}, \mathbf{ID} \rangle$ to output the discrete logarithm relations for $G, \mathbf{ID}$ and the witness $\mathbf{a}$. Yuen et al. have proved that such an extractor exists [33]. Since $C = xG_{pk} \oplus G_{id} = xsG \oplus G_{id}$, we have that $xsG \oplus G_{id} = x \langle \mathbf{a}, \mathbf{s} \rangle G \oplus \langle \mathbf{a}, \mathbf{ID} \rangle$.

For the protocol of aggregated public keys and identities argument, we use the extractor EK_{DL} to obtain a witness s and utilize the extractor EK_{semi} twice to obtain two witness $\mathbf{a}'$ and $\mathbf{a}''$ at the two different challenges x' and x'' . Then, we have

$$(x' - x'')sG = \langle x'\mathbf{a}' - x''\mathbf{a}'', \mathbf{s} \rangle G,$$

and

$$(x' - x'')G_{id} = \langle x'\mathbf{a}' - x''\mathbf{a}'', \mathbf{ID} \rangle.$$

If we cannot find a non-trivial discrete logarithm relation for G and $\mathbf{ID}$, then we have $\mathbf{a}' = \mathbf{a}''$ and $s = \langle \mathbf{a}', \mathbf{s} \rangle$. Hence, $G_{id} = \langle \mathbf{a}', \mathbf{ID} \rangle$ and $G_{pk} = sG = \langle \mathbf{a}', \mathbf{s} \rangle G$. For the protocol of aggregated public keys and identities argument, the following equations hold for x' and x'' .

$$x^s G_{pk} \oplus G_{id} = \langle \mathbf{a}^s, \mathbf{s} \rangle xG \oplus \langle \mathbf{a}^s, \mathbf{ID} \rangle, \varsigma = x' \text{ or } x''$$

And then, we can compute

$$\mathbf{a}_G = \frac{x'\mathbf{a}' - x''\mathbf{a}''}{x' - x''},$$

$$s_G = \frac{\mathbf{a}' - \mathbf{a}''}{x' - x''},$$

and we have $G_{pk} = s_G G, G_{id} = \langle \mathbf{a}_G, \mathbf{ID} \rangle$. Thus, if the discrete logarithm assumption holds, we obtain

$$x^s \langle \mathbf{a}^s, \mathbf{s} \rangle = x^s s_G, \mathbf{a}^s = \mathbf{a}_G.$$

From the above equations, we have $x^s \langle \mathbf{a}_G, \mathbf{s} \rangle = x^s s_G$, and thus $\langle \mathbf{a}_G, \mathbf{s} \rangle = s_G$. Consequently, let $\mathbf{a} = \mathbf{a}_G$ and $s = s_G$, we get a valid witness of the aggregated public keys and identities argument.

C. Proving the Subgroup Size

In our construction, the prover $\mathcal{P}$ must convince the verifier $\mathcal{V}$ that the subgroup size of participating signers $t = \langle \mathbf{d}, \mathbf{d} \rangle$ is correct, where $\mathbf{d} \in \{0, 1\}^n$ is a bit vector indicating participation ($d_i = 1$ if the i -th signer participates, and 0 otherwise). To achieve this, we employ the Univariate SumCheck technique introduced by Sourav et al. [26].

Let μ be a fixed element of Z_q such that the set $U = \{\mu, \mu^2, \dots, \mu^n\}$ consists of distinct elements. Define $z_U(x) = \prod_{i=1}^n (x - \mu^i)$, the vanishing polynomial over U . Given a polynomial $d(x) \in Z_q[x]$ of degree at most $n - 1$ that satisfies $d(\mu^i) = d_i$ for all $i \in [n]$, the following lemma holds.

Lemma 1 (Univariate SumCheck [26]). For any polynomial $d(x)$ of degree $n - 1$, there exist unique polynomials $q_d(x)$ and $r_d(x)$ such that

$$d(x) \cdot d(x) = q_d(x) \cdot z_U(x) + x \cdot r_d(x) + t \cdot n^{-1},$$

where $t = \sum_{i=1}^n d_i$ and n^{-1} is the multiplicative inverse of n in Z_q . Moreover, for any $\tau \in Z_q$, the value $d(\tau)$ can be computed from the coefficients of $d(x)$.

Based on this lemma, the prover $\mathcal{P}$ can construct a succinct proof for the correct value of t without revealing $\mathbf{d}$. The proof generation and verification proceed as follows.

Proof generation: $\mathcal{P}$ first constructs the polynomial $d(x)$ of degree $n - 1$ such that $d(\mu^i) = d_i$ (e.g., via interpolation). It then computes the unique polynomials $q_d(x)$ and $r_d(x)$ satisfying the identity in Lemma 1. Let $p_d(x) = x \cdot r_d(x) + t \cdot n^{-1}$. $\mathcal{P}$ chooses a random $\tau \in Z_q$ (which can be a public challenge or part of the transcript) and computes the commitments

$$P_d = d(\tau)G, \quad Q = q_d(\tau)G, \quad R = r_d(\tau)G, \quad P = p_d(\tau)G.$$

The proof is $\pi = \{Q, R, P\}$.

Proof verification: Upon receiving π , the verifier $\mathcal{V}$ checks the following two pairing equations:

$$e(P_d, P_d) = e(Q, z_U(\tau)G) \cdot e(R, \tau G) \cdot e(tG, n^{-1}G),$$

$$e(P, G) = e(R, \tau G) \cdot e(tG, n^{-1}G).$$

If both hold, $\mathcal{V}$ accepts that the claimed subgroup size t is correct.

Correctness: If the prover follows the protocol honestly, then by Lemma 1 we have

$$d(\tau) \cdot d(\tau) = q_d(\tau) \cdot z_U(\tau) + \tau \cdot r_d(\tau) + t \cdot n^{-1}.$$

Multiplying both sides by the generator G and applying the bilinear map yields

$$e(d(\tau)G, d(\tau)G) = e(q_d(\tau)G, z_U(\tau)G) \cdot e(r_d(\tau)G, \tau G) \cdot e(tG, n^{-1}G).$$

Thus the first verification equation holds. The second equation follows from the definition $p_d(x) = xr_d(x) + t \cdot n^{-1}$ and the fact that the bilinear map respects group addition.

Soundness: Suppose an adversary produces a proof π that satisfies both verification equations for a claimed t . From the first equation and the bilinearity, we have

$$e(d(\tau)G, d(\tau)G) = e(q_d(\tau)G, z_U(\tau)G) \cdot e(r_d(\tau)G, \tau G) \cdot e(tG, n^{-1}G).$$

If we define $\tilde{d} = d(\tau)$, $\tilde{q} = q_d(\tau)$, $\tilde{r} = r_d(\tau)$, and $\tilde{p} = p_d(\tau)$, then the second equation gives $\tilde{p}G = \tilde{r}\tau G + t\tilde{d}G$. The first equation, together with the fact that the vanishing polynomial $z_U(\tau)$ is non-zero for a random τ with overwhelming probability (unless τ coincides with a root), implies that the polynomial identity holds at a random point τ , and thus by the Schwartz-Zippel lemma [39], the polynomial relation holds over Z_q with high probability. Consequently, the extracted polynomial $d(x)$ must satisfy the lemma, and the computed t equals $\sum_{i=1}^n d_i$. Hence, the protocol is sound.

The above proof is succinct, requiring only three group elements and two pairing equations, and it does not reveal the underlying bit vector $\mathbf{d}$. In the context of our multi-signature scheme, this proof is used in the *Combine* and *Verimul* algorithms to guarantee that the subgroup size meets the required threshold without disclosing the identities of the participating signers.

Notation Summary: Table I summarizes key notation used in our construction.

D. Definition of Certificateless Subgroup Multi-Signature

In the certificateless subgroup multi-signature scheme, there exists a KGC responsible for generating the partial private key for each signer's identity. Assume that there are n users in the system, and the set of their identities is denoted as $\mathcal{ID} = \{ID_1, \dots, ID_n\}$. For each signing process, there exists an aggregator within the subgroup of participating signers. A certificateless subgroup multi-signature scheme

TABLE I
KEY NOTATION

Symbol	Meaning
a	Selector vector (hides participant identities)
s	Secret key vector
d	Bit vector indicating participation
G_{pk}	Aggregated public key
G_{id}	Aggregated identity
t	Subgroup size (number of signers)
t'	Predefined threshold value
pk	Vector of public keys
ID	Vector of hashed identities

usually includes eight probabilistic polynomial-time algorithms: *Setup*, *KeyGen*, *Extract*, *PSign*, *PVerify*, *Combine*, *KeyAgg*, *Verimul*. The *Setup* algorithm is responsible for generating the system parameters. The *KeyGen* algorithm is designed for generating the public/secret key pair for each signer. The *Extract* algorithm is used for extracting the private key from the signer's identity. Each signer utilizes the *PSign* algorithm to generate its individual signature, and each individual signature can be verified by the *PVerify* algorithm. The *Combine* algorithm is designed for combining all individual signatures from the participating signers, and producing the multi-signature. The *KeyAgg* algorithm is used for aggregating the participating signers' public keys and identities where the set of participating signers is a subgroup of the whole set of signers. The *Verimul* algorithm is responsible for verifying the multi-signature. Assume that the indices set of participating signers is $I \subset \{1, \dots, n\}$. The formalized certificateless multi-signature scheme has the following interfaces:

Setup(1^λ): This algorithm takes the security parameter λ as input, and outputs the primary secret key msk and the public system parameters pp including the main public key mpk .

KeyGen(pp, n): The algorithm takes the system parameters pp and the total number of signers n as input, outputs (s_i, pk_i) as the secret/public key pair for the i th signer, $i \in \{1, \dots, n\}$. The vector of public keys is denoted as $\mathbf{pk} = (pk_1, \dots, pk_n)$.

Extract($pp, \mathcal{ID}, msk$): This algorithm takes the system parameters pp , the set of signers' identities $\mathcal{ID}$ and the primary secret key msk as input, and outputs d_{ID_i} as the private key corresponding to the identity ID_i for the i th signer, $i \in \{1, \dots, n\}$.

PSign(pp, m, d_{ID_i}, s_i): The i th signer uses this algorithm with the system parameters pp , its private key d_{ID_i} and secret key s_i to generate an individual signature σ_i .

PVerify($pp, m, \sigma_i, pk_i, ID_i$): This algorithm takes the system parameters pp , a message m , the i th signer's public key pk_i and identity ID_i as input, and checks whether σ_i is generated using the secret key s_i and private key d_{ID_i} .

Combine($pp, I, \{\sigma_i\}_{i \in I}, t, t'$): The aggregator firstly checks if the claimed subgroup size t meets the threshold t' , aborts and outputs $\perp$ if $t < t'$. And then, the aggregator uses this algorithm with the system parameters pp , the indices set of participating signers I , the set of submitted individual signatures $\{\sigma_i\}_{i \in I}$, and the subgroup size t of participating signers, to generate an aggregated multi-signature σ .

KeyAgg($pp, I, \mathbf{pk}, \mathcal{ID}$): The aggregator utilizes the algorithm with the system parameters pp , the indices set of participating signers I , the vector of public keys $\mathbf{pk}$ and the set of identities $\mathcal{ID}$, to generate an aggregated public key apk .

Verimul(pp, m, σ, t', apk): This algorithm outputs 1 only if the message m is signed by the participating signers and the subgroup size of participating signers is larger than t' , where t' is a given threshold value.

A certificateless multi-signature scheme should satisfy the following correctness and unforgeability property.

Correctness: For any n and any subset $I \subseteq \{1, \dots, n\}$, an honestly generated individual signature should always be valid. Additionally, a multi-signature generated by any arbitrary subgroup should pass the verification process. The correctness of a certificateless multi-signature scheme can be formally defined as follows:

$$Pr[PVerify(pp, m, PSign(m, d_{ID_i}, s_i), pk_i, ID_i)] = 1,$$

$$Pr[Verimul(pp, I, Combine(I, \{\sigma_i\}_{i \in I}, \mathbf{pk}, \mathcal{ID}), apk)] = 1.$$

Unforgeability: The unforgeability of a certificateless subgroup multi-signature scheme is defined through two games between a challenger $\mathcal{C}$ and two adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$. Here, $\mathcal{A}_1$ models a malicious user who can arbitrarily replace the public key of any signer (i.e., choose a new public key without providing the corresponding secret key). This reflects the lack of certification in certificateless systems. $\mathcal{A}_1$ may also corrupt users by obtaining their partial private keys (via Extract queries) and their secret keys (via Secret key queries), but cannot replace a users public key and simultaneously extract its partial private key for the same identity unless the replacement is allowed in the game. $\mathcal{A}_2$ models a malicious Key Generation Center (KGC) who knows the primary secret key msk and can generate partial private keys for any identity, but cannot replace the public keys of honest users (since public key replacement is under the control of users, not the KGC). This captures the scenario where the KGC may attempt to impersonate users without knowing their self-generated secret keys.

Game 1 (between $\mathcal{C}$ and $\mathcal{A}_1$):

Setup: $\mathcal{C}$ runs the Setup algorithm to generate the system parameters pp and the primary secret key msk , and sends pp to $\mathcal{A}_1$.

Hash queries: $\mathcal{A}_1$ can make hash queries for all hash oracles in the certificateless subgroup multi-signature scheme.

Extract queries: When $\mathcal{A}_1$ makes an Extract query on the identity ID_i , $\mathcal{C}$ runs the Extract algorithm, and responses the private key d_{ID_i} to $\mathcal{A}_1$.

Secret key queries: $\mathcal{C}$ maintains a list L_s , initially empty. Each entry is of the form $(ID_i, s_i, pk_i, flag_i)$, where $flag_i \in \{"original", "replaced"\}$. When $\mathcal{A}_1$ makes a secret key query on the identity ID_i , if the entry is found and $flag_i = "original"$, then $\mathcal{C}$ returns s_i to $\mathcal{A}_1$. If the entry is found and $flag_i = "replaced"$, then $\mathcal{C}$ returns $\perp$ (since $\mathcal{C}$ does not know the secret key corresponding to the adversary-chosen public key); otherwise, it runs the KeyGen algorithm to

generate a secret/public key pair (s_i, pk_i) for $\mathcal{A}_1$ and adds $(ID_i, s_i, pk_i, "original")$ to L_s .

Public key replacing queries: $\mathcal{A}_1$ can use (s_i, pk_i) chosen by itself to replace the signer ID_i 's key pair. $\mathcal{C}$ searches L_s for an entry for ID_i and if the entry is found then it updates the entry as $pk_i = pk'_i$ and $flag_i = "replaced"$, otherwise it aborts.

Signature queries: When $\mathcal{A}_1$ make a signature query on a message m for an identity ID_i and a public key pk_i , $\mathcal{C}$ runs the PSign algorithm to generate the individual signature σ_i for it. **Output:** $\mathcal{A}_1$ outputs a multi-signature σ^* for a message m^* and a set of participating signers I^* .

If $\mathcal{A}_1$ satisfies the following two conditions then it wins the above game.

- 1) There exists at least one identity $ID^* \in I^*$ which has not been queried for the Extract and the Public key replacing simultaneously,
- 2) m^* has not been made the signature queries for ID^* , and $Verimul(pp, m^*, I^*, \sigma^*, apk^*) = 1$.

Game 2 (between $\mathcal{C}$ and $\mathcal{A}_2$):

Setup: $\mathcal{C}$ runs the Setup algorithm to generate the system parameters pp and the primary secret key msk , and sends pp to $\mathcal{A}_2$.

Hash queries: $\mathcal{A}_2$ can make hash queries for all hash oracles in the certificateless subgroup multi-signature scheme.

Secret key queries: When $\mathcal{A}_2$ makes a secret key query on the identity ID_i , if ID_i has been queried, then $\mathcal{C}$ returns $\perp$; otherwise, it runs the KeyGen algorithm to generate a secret/public key pair (s_i, pk_i) for $\mathcal{A}_2$.

Signature queries: When $\mathcal{A}_2$ make a signature query on a message m for an identity ID_i , $\mathcal{C}$ runs the PSign algorithm to generate the individual signature σ_i for it.

Output: $\mathcal{A}_2$ outputs a multi-signature σ^* for a message m^* and a set of participating signers I^* .

If the following conditions are satisfied, $\mathcal{A}_2$ wins the above game:

- 1) m^* has not been made the signature queries for any $pk_i \in I^*$.
- 2) For at least one identity $ID^* \in I^*$, the secret key corresponding to ID^* has not been queried by $\mathcal{A}_2$ through the secret key query oracle.
- 3) $Verimul(pp, m^*, I^*, \sigma^*, apk^*) = 1$.

If the probabilities of $\mathcal{A}_1$ winning the Game 1 and $\mathcal{A}_2$ winning the Game 2 are both negligible, then we say that the certificateless subgroup multi-signature scheme is unforgeable under the adaptively chosen message attacks.

Our security model (Game 1 and Game 2) captures two classical adversarial settings: a malicious user who may replace public keys (Game 1) and a malicious KGC who controls the primary secret key (Game 2). In distributed consensus environments, additional adversarial behaviors such as collusion and denial-of-service (DoS) attacks are also relevant. Collusion

¹ $\mathcal{A}_2$ represents a malicious KGC who does not need to make the Extract queries and cannot make the Public key replacing queries.

among multiple signers is inherently limited by the unforgeability guarantee: even if a subset of signers colludes, they cannot forge a signature on behalf of an honest signer unless they collectively control a threshold of secret keys, which is prevented by the hardness of the CDH assumption and the binding property of the inner product argument. DoS attacks, where adversaries submit malformed or numerous partial signatures, are mitigated by the lightweight per-signature verification (PVerify) and the fact that the aggregator can discard invalid contributions before aggregation.

E. Construction of Certificateless Subgroup Multi-Signature

Our certificateless subgroup multi-signature scheme is based on the BLS signature [35] and Sakai's identity-based signature [37]. It utilizes the inner product argument (aggregated public keys and identities argument) to enable arbitrary subgroups of signers to generate a multi-signature, provided that the subgroup size exceeds a given threshold value. In our certificateless multi-signature scheme, the identities of all participating signers are kept anonymous to the verifier, thereby protecting the privacy of the signers in certain applications. The verification process does not require the public keys or identities of the participating signers; instead, the verifier generates the aggregated public key independently. The aggregated public keys and identities argument is then used to authenticate the aggregated public key. Assume that there are n signers in the system, with their corresponding identities denoted as $\mathcal{ID} = \{ID_1, \dots, ID_n\}$. In our certificateless multi-signature scheme, the aggregator acts as the prover $\mathcal{P}$ in the aggregated public keys and identities argument, while the multi-signature verifier acts as the verifier $\mathcal{V}$. Since the aggregated public key is directly used in the Verimul algorithm of our scheme and is not authenticated beforehand, the aggregated public keys and identities argument is crucial for proving the correctness of the aggregated public key. The certificateless multi-signature scheme is consisted of the following eight probabilistic polynomial-time algorithms.

Setup(1^λ): Let $(q, \mathbb{G}, \mathbb{G}_T, e, G)$ be a bilinear group with order q , and $G \in \mathbb{G}$. This algorithm chooses $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}$, $H_2 : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ and $H_3 : \{0, 1\}^* \rightarrow \mathbb{G}$ as three cryptographic hash functions. It selects $\mu \in \mathbb{Z}_q$ and sets $U = \{\mu, \mu^2, \dots, \mu^n\}$. KGC chooses $x \in \mathbb{Z}_q$ as the primary secret key msk and publishes $Y = xG$ as the main public key. The algorithm outputs the system parameters $pp = (q, \mathbb{G}, \mathbb{G}_T, e, G, H_1, H_2, H_3, U, Y)$.

KeyGen(pp, n): The i th signer chooses $s_i \in \mathbb{Z}_q$ as its secret key and sets $pk_i = s_i \cdot G$ as its public key. For $i \in \{1, \dots, n\}$. The vector of public keys is denoted as $\mathbf{pk} = (pk_1, \dots, pk_n)$.

Extract($pp, \mathcal{ID}, msk$): KGC computes $d_{ID_i} = x \cdot H_1(ID_i)$ as the i th signer's private key for all $ID_i \in \mathcal{ID}$.

PSign(pp, m, d_{ID_i}, s_i): The i th signer randomly chooses $r_i \in \mathbb{Z}_p$, and computes $\sigma_{i1} = (r_i + s_i)H_3(m) \oplus d_{ID_i}$ and $\sigma_{i2} = r_i G$. Thus, its individual signature is $\sigma_i = (\sigma_{i1}, \sigma_{i2})$.

PVerify($pp, m, \sigma_i, pk_i, ID_i$): The verifier checks whether the following equation holds:

$$e(\sigma_{i1}, G) = e(H_3(m), \sigma_{i2})e(H_3(m), pk_i)e(H_1(ID_i), Y).$$

If it holds, then the i th signer's individual signature is valid.

Combine($pp, t, t', \{\sigma_i\}_{i \in I}, \mathbf{pk}, \mathcal{ID}$): This algorithm can be divided into the following phases:

- 1) $\mathcal{P}$ checks if the claimed subgroup size t meets the threshold t' , aborts and outputs $\perp$ if $t < t'$.² And then, $\mathcal{P}$ checks whether the total number of submitted individual signatures $|I|$ equals the subgroup size of participating signers t . And then, $\mathcal{P}$ sets a bit vector $\mathbf{d} = (d_1, \dots, d_n)$ where $d_i = 1$ for $i \in I$ otherwise $d_i = 0$.
- 2) For the set of participating signers I , $\mathcal{P}$ sets a vector $\mathbf{a}_I = (a_1, \dots, a_n)$ where $a_i = 0$ if $i \notin I$ otherwise $a_i = H_2(ID_i, pk_i, \mathbf{pk}, \mathcal{ID})$. Then, $\mathcal{P}$ constructs a polynomial $d(x)$ such that $d(\mu^i) = d_i$,³ and computes two polynomials $q_d(x)$ and $r_d(x)$ such that $d(x) \cdot d(x) = q_d(x)z_U(x) + xr_d(x) + t \cdot n^{-1}$. It chooses a random $\tau \in \mathbb{Z}_q$ and computes $P_d = d(\tau)$ as a commitment. Let $p_d(x) = xr_d(x) + t \cdot n^{-1}$, it outputs the proof as $\pi = \{q_d(\tau)G, r_d(\tau)G, p_d(\tau)G\}$.
- 3) $\mathcal{P}$ computes $\sigma_1 = \oplus_{i \in I} a_i \sigma_{i1}$ and $\sigma_2 = \oplus_{i \in I} a_i \sigma_{i2}$.
- 4) $\mathcal{P}$ computes $D = \langle \mathbf{a}, \mathbf{d} \rangle G$.

$\mathcal{P}$ outputs the signature as $\sigma = (\sigma_1, \sigma_2, P_d, \pi, D, t)$ which includes the aggregated multi-signature and the proof π of subgroup size t .

KeyAgg($pp, I, \mathbf{pk}, \mathcal{ID}$): This algorithm can be divided into the following phases:

- 1) For the set of participating signers I , $\mathcal{P}$ sets the vector of hash values $\mathbf{a}_I$.
- 2) $\mathcal{P}$ sets a vector of hashed identity as $\mathbf{ID} = (H_1(ID_1), \dots, H_1(ID_n))$ for the whole set of signers.
- 3) $\mathcal{P}$ computes computes two inner products between the vector of field elements and the vector of group elements as $G_{pk} = \langle \mathbf{a}_I, \mathbf{PK} \rangle$ and $G_{id} = \langle \mathbf{a}_I, \mathbf{ID} \rangle$. The aggregated public key is $apk = (G_{pk}, G_{id})$.

Verimul(pp, m, σ, t', apk): This algorithm can be divided into the following phases:

- 1) The verifier $\mathcal{V}$ and $\mathcal{P}$ (with the help of participating signers) execute the aggregated public keys and identities argument on input as $(G_{pk}, G_{id}, G, D : \mathbf{a}_I, s_I, \mathbf{d})$ where $s_I = (s_1, \dots, s_n)$ is the vector of secret keys for the participating signers and sets $s_i = 0$ when $i \notin I$. If the aggregated public keys and identities argument outputs *True*, then proceed to step 2.
- 2) If the aggregated public keys and identities argument outputs *True*, the verifier $\mathcal{V}$ checks whether $e(\sigma_1, G) = e(H_3(m), \sigma_2)e(H_3(m), G_{pk})e(G_{id}, y)$ holds. If the equation holds, then σ is a valid multi-signature with respect to the aggregate public key apk .

² In the Combine algorithm, we introduce a preliminary check (Step 1) comparing the claimed subgroup size t with the given threshold t' . This is a lightweight sanity check to avoid unnecessary computation for requests that are clearly non-compliant. The definitive and fraud-proof verification of the participant count and its compliance with t' is still performed by the verifier in the Verimul algorithm via the zero-knowledge SumCheck protocol, which forms the basis of our security proofs.

³ Given $U = \{\mu, \mu^2, \dots, \mu^n\}$, it is very efficient to construct a polynomial $d(x)$ such that $d(\mu^i) = d_i$ which is mentioned in Sourav et al.'s scheme, and we don't discuss it in this paper.

- 3) $\mathcal{V}$ checks whether $e(d(\tau)G, P_d) = e(q_d(\tau)G, z_U(\tau))e(r_d(\tau)G, \tau G)e(tG, 1/nG)$ and $e(p_d(\tau)G, G) = e(r_d(\tau)G, \tau G)e(tG, 1/nG)$ hold. If they hold, then $\mathcal{V}$ accepts the subgroup size t . Then, $\mathcal{V}$ checks whether $t \geq t'$ where t' is a pre-defined threshold value. If $t < t'$, then $\mathcal{V}$ rejects σ .

F. Security Analysis for Our Scheme

Correctness: For the PVerify algorithm, we have

$$\begin{aligned} e(\sigma_{i1}, G) &= e((r_i + s_i)H_3(m) \oplus d_{ID_i}, G) \\ &= e((r_i + s_i)H_3(m), G)e(d_{ID_i}, G) \\ &= e(r_i H_3(m), G)e(s_i H_3(m), G)e(d_{ID_i}, G) \\ &= e(H_3(m), \sigma_{i2})e(H_3(m), pk_i)e(H(ID_i), Y). \end{aligned}$$

For the Verimul algorithm, we have

$$\begin{aligned} e(\sigma_1, G) &= e(\oplus_{i \in I} \sigma_{i1}, G) \\ &= e\left(\sum_{i \in I} a_i((r_i + s_i)H_3(m) \oplus d_{ID_i}), G\right) \\ &= e\left(\sum_{i \in I} a_i r_i H_3(m), G\right) e\left(\sum_{i \in I} a_i s_i H_3(m), G\right) \\ &\quad \times e(\oplus_{i \in I} a_i d_{ID_i}, G) \\ &= e(H_3(m), \sigma_2)e(H_3(m), \oplus_{i \in I} a_i pk_i)e(\oplus_{i \in I} a_i H(ID_i), Y) \\ &= e(H_3(m), \sigma_2)e(H_3(m), G_{pk})e(G_{id}, Y). \end{aligned}$$

Theorem 2. Our certificateless subgroup multi-signature scheme is unforgeable under the random oracles if the CDH assumption holds over the group $\mathbb{G}$ and BLS signature is unforgeable

Proof. Suppose $\mathcal{A}_1$ and $\mathcal{A}_2$ be two forgers against our certificateless subgroup multi-signature scheme who represents two adversaries in the security model. $\mathcal{B}$ is a challenger who acts as an adversary to the BLS signature using $\mathcal{A}_1$ and $\mathcal{A}_2$ as subroutines. Let $\mathcal{F}$ be a challenger for $\mathcal{A}_1$ and $\mathcal{A}_2$, who is against the CDH assumption. Let H_1, H_2 be two random oracles. The challenger $\mathcal{B}$'s goal is to output a valid BLS signature. Given a CDH instance $(\mathbb{G}, \mathbb{G}_T, q, G, \alpha G, \beta G)$ where $G \in \mathbb{G}, \alpha, \beta \leftarrow_R Z_q$, the goal of $\mathcal{F}$ is to find $\alpha\beta G$ to the instance by using $\mathcal{B}$ as subroutine.

For the adversary $\mathcal{A}_1$, $\mathcal{B}$ outputs $y = \alpha G$ as the main public key which implies that α is the primary secret key. $\mathcal{B}$ assumes that ID^* is the challenge identity. $\mathcal{B}$ should answers the individual signature queries and hash queries from $\mathcal{A}_1$. $\mathcal{B}$ sends $pp = (q, \mathbb{G}, \mathbb{G}_T, e, G, H_1, H_2, U, Y)$ as the system parameters to $\mathcal{A}_1$.

For the adversary $\mathcal{A}_2$, $\mathcal{B}$ chooses $x \in Z_q$ as the primary secret key and computes $Y = xG$ as the main public key. $\mathcal{B}$ assumes that ID^* is the challenge identity, and answers the individual signature queries and hash queries from $\mathcal{A}_2$. $\mathcal{B}$ sends $pp = (q, \mathbb{G}, \mathbb{G}_T, e, G, H_1, H_2, H_3, U, Y)$ as the system parameters and x as the primary secret key to $\mathcal{A}_2$.

We define the games between $\mathcal{B}$ and $\mathcal{A}_1, \mathcal{A}_2$ as follows. For simplicity, we combine the Game 1 and Game 2.

- Assume that $\mathcal{B}$ runs $\mathcal{A}_1$ on the vector of randomness $f = (o, o_1, \dots, o_{q_H})$ for the H_2 oracle.
- H_1 queries: The hash oracle H_1 is answered by $\mathcal{B}$ as follows. Firstly, $\mathcal{B}$ initialises a list of quadruples $L_{H_1} = \{(\rho, c, H)\}$ with $\emptyset$. And then, for a request of H_1 query on ρ from $\mathcal{A}_1$ or $\mathcal{A}_2$, $\mathcal{B}$ searches L_{H_1} and outputs H if (ρ, c, H) exists in L_{H_1} . Otherwise, it responses with $H = cG$ where $c \leftarrow_R Z_q$. It updates $L_{H_1} = L_{H_1} \cup \{(\rho, c, H)\}$ after each querying.
- Extract queries from $\mathcal{A}_1$: For the Extract query on ID from $\mathcal{A}_1$ where $ID \neq ID^*$, $\mathcal{B}$ firstly executes the H_1 query as above, and gets c . And then it responses $d_{ID} = cY$ to $\mathcal{A}_1$ as the private key for ID . Since $H_1(ID) = cG$, then $d_{ID} = xH_1(ID) = xcG = cY$ is a valid private key. If $ID = ID^*$, then $\mathcal{B}$ responses $H_1(ID^*) = \beta G$.
- H_2 queries $\mathcal{A}_1$: $\mathcal{B}$ maintains a list $L_{H_2} = \{(\rho, o)\}$ and initialises it with $\emptyset$. When $\mathcal{A}_1$ makes a query on $\rho = (ID_i, pk_i, \mathbf{pk}, \mathcal{ID})$, $\mathcal{B}$ responses o if (ρ, o) exists; otherwise it responses with c_i if $ID_i = ID^*$ and $i \in I$; otherwise it responses with $o \leftarrow_R Z_q$ if $i \in I$; otherwise it responses $\perp$. $\mathcal{B}$ updates $L_{H_2} = L_{H_2} \cup \{(\rho, o)\}$ after each querying.
- H_2 queries $\mathcal{A}_2$: $\mathcal{B}$ maintains a list $L_{H_2} = \{(\rho, o)\}$ and initialises it with $\emptyset$. When $\mathcal{A}_2$ makes a query on $\rho = (ID_i, pk_i, \mathbf{pk}, \mathcal{ID})$, $\mathcal{B}$ responses with o and $o \leftarrow_R Z_q$ if $i \in I$, otherwise it responses $\perp$. $\mathcal{B}$ updates $L_{H_2} = L_{H_2} \cup \{(\rho, o)\}$ after each querying.
- H_3 queries: $\mathcal{B}$ defines a list of tuples $L_{H_3} = \{(\rho, \lambda, M)\}$ and initialises it with $\emptyset$. For the H_3 queries on ρ from $\mathcal{A}_1$ or $\mathcal{A}_2$, $\mathcal{B}$ responses M if (ρ, λ, M) exists in L_{H_3} . Otherwise, it randomly chooses $\lambda \in Z_q$ and computes $M = \lambda G$ as the response. $\mathcal{B}$ updates $L_{H_3} = L_{H_3} \cup \{(\rho, \lambda, M)\}$ after each querying.
- Secret key queries: $\mathcal{B}$ defines a list of tuples $L_s = \{(ID, s, pk, flag)\}$, and initialises it with $\emptyset$. When $\mathcal{A}_1$ or $\mathcal{A}_2$ makes a secret key query on ID_i , if $(ID_i, s_i, pk_i, flag_i)$ exists in L_s and $flag_i = \text{"original"}$, then $\mathcal{B}$ responses s_i ; if the entry exists and $flag_i = \text{"replaced"}$, $\mathcal{B}$ returns $\perp$ (since the challenger does not know the secret key corresponding to an adversarially chosen public key). If the entry is not found and $ID_i \neq ID^*$, $\mathcal{B}$ randomly chooses $s_i \in Z_q$, computes $pk_i = s_i G$, returns s_i and adds $(ID_i, s_i, pk_i, \text{"original"})$ to L_s . If $ID_i = ID^*$, $\mathcal{B}$ returns $\perp$ and sets $pk_i = \beta G$ (the CDH challenge), with $s_i = \perp$.
- Public key replacing queries: Upon receiving a public key replacing request (ID_i, pk'_i) from $\mathcal{A}_1$, $\mathcal{B}$ searches L_s for an entry $(ID_i, s_i, pk_i, flag_i)$. If the entry exists and $ID_i \neq ID^*$, $\mathcal{B}$ updates the entry to $pk_i = pk'_i$, $s_i = \perp$, and sets $flag_i = \text{"replaced"}$. If the entry does not exist or $ID_i = ID^*$, $\mathcal{B}$ aborts the game (as it cannot replace the challenge identity's key without breaking the simulation). This ensures that $\mathcal{A}_1$ cannot simultaneously replace the public key of a target user and obtain its partial private key, which would trivialize forgery.
- Signature queries: Upon receiving the individual signature queries for a message m and an identity ID , $\mathcal{B}$ response $\perp$

if $ID = ID^*$. Otherwise, it searches the list L_{H_1} for ID , gets H and the corresponding private key d_{ID} . Following that, $\mathcal{B}$ searches L_s to get the secret key s corresponding to ID . And then, $\mathcal{B}$ searches L_{H_3} for m , and gets M . Finally, $\mathcal{B}$ selects $\delta \in Z_q$ and computes $\varrho_1 = (\delta + s)M + d_{ID}$ and $\varrho_2 = \delta G$ as the response.

Finally, a aggregate public key $apk = (G_{pk}, G_{id})$, a threshold value t' and a forged multi-signature σ^* for the message m^* are generated by $\mathcal{A}_1$ or $\mathcal{A}_2$. (m^*, ID^*) can not be made the signature query, but the multi-signature σ^* is valid. Let $ID_k = ID^*$ where k is the index of ID^* in the set of identities.

Assume that j_f is the index of $H_2(ID^*, pk^*, \mathbf{pk}, \mathcal{ID})$ in the vector of randomness f that $H_2(ID^*, pk^*, \mathbf{pk}, \mathcal{ID}) = o_{j_f}$. Thus, $\mathcal{B}$ gets the final output from the adversary $\mathcal{A}_1$ which can be denote as $j_f, \{\sigma^*, apk\}$. By using knowledge soundness of the aggregated public keys and identities argument protocol, the vector of field elements a can be extracted such that $G_{pk} = \langle \mathbf{a}, \mathbf{pk} \rangle$ and $G_{id} = \langle \mathbf{a}, \mathbf{ID} \rangle$. Let $\mathbf{b}$ be the corresponding bit vector, and I be the set of indices such that $b_i = 1$. $\mathcal{B}$ can determine whether the index of ID^* belongs to I . If it is not, then the game is failure.

The challenger $\mathcal{F}$ can rewind $\mathcal{A}_1$'s algorithm through $\mathcal{B}$ twice for the same set I of participating signers by using the Forking Lemma, and gets the adversary's two outputs from $\mathcal{B}$ that are $\{j_f, \{\sigma^*, apk\}\}$ and $j_f, \{\sigma'^*, apk'\}$. By using knowledge soundness of the aggregated public keys and identities argument protocol, two corresponding vectors $\mathbf{a}$ and $\mathbf{a}'$ can be obtained. The vectors of randomness f and f' are the same in the two rounds except for o_{j_f} which we denote as $f|_{j_f} = f'|_{j_f}$. We have $a_k = o_{j_f}$ and $a'_k = o'_{j_f}$, and $a_i = a'_i$ for $i \in I, i \neq k$. Thus, $G_{pk} - G'_{pk} = (a_k - a'_k)pk_k$ and $G_{id} - G'_{id} = (a_k - a'_k)H_1(ID_k)$. Since two forged multi-signatures σ^* and σ'^* are valid, we have that

$$\begin{aligned} e(\sigma_1^*, G) &= e(H_3(m^*), \sigma_2^*)e(H_3(m^*), G_{pk})e(G_{id}, Y), \\ e(\sigma_1'^*, G) &= e(H_3(m^*), \sigma_2'^*)e(H_3(m^*), G'_{pk})e(G'_{id}, Y), \end{aligned}$$

and

$$\begin{aligned} e(\sigma_1^* - \sigma_1'^*, G) &= e(H_3(m^*), \sigma_2^* - \sigma_2'^*) \cdot \\ &e(H_3(m^*), G_{pk} - G'_{pk})e(G_{id} - G'_{id}, Y) \\ &= e(\lambda^* G, \sigma_2^* - \sigma_2'^*)e(\lambda^* G, (a_k - a'_k)pk_k) \\ &e((a_k - a'_k)H_1(ID^*), Y) \\ &= e(\lambda^* G, \sigma_2^* - \sigma_2'^*)e(\lambda^* G, (a_k - a'_k)\beta G) \cdot \\ &e((a_k - a'_k)\beta G, \alpha G). \\ &= e(\lambda^* G, \sigma_2^* - \sigma_2'^*)e((a_k - a'_k)(\lambda^* \beta + \alpha \beta)G, G) \end{aligned}$$

We have

$$(\sigma_1^* - \sigma_1'^*) - \lambda^*(\sigma_2^* - \sigma_2'^*) = (a_k - a'_k)(\lambda^* \beta + \alpha \beta)G,$$

and thus,

$$\alpha \beta G = \frac{(\sigma_1^* - \sigma_1'^*) - \lambda^*(\sigma_2^* - \sigma_2'^*) - (a_k - a'_k)\lambda^* \beta G}{a_k - a'_k}.$$

Therefore, the challenger $\mathcal{F}$ can solve the CDH assumption by using $\mathcal{A}_1$'s algorithm through $\mathcal{B}$, and thus our certificateless subgroup multi-signature scheme is unforgeable against the adversary $\mathcal{A}_1$.

For the adversary $\mathcal{A}_2$, $\mathcal{B}$ can construct a valid BLS signature from its forged multi-signature σ^* under the aggregate public key $apk = (G_{pk}, G_{id})$. By using knowledge soundness of the aggregated public keys and identities argument protocol, the corresponding vectors $\mathbf{a}$ can be obtained. Let $ID_k = ID^*$ and $k \in I$. $\mathcal{B}$ can compute $\sigma_{BLS} = a_k^{-1} \cdot (\sigma_1^* - \sum_{i \in I, i \neq k} a_i \sigma_{i1}) - d_{ID^*}$ and $pk_{BLS} = pk_k + a_k^{-1} \cdot (\sigma_2^* - \sum_{i \in I, i \neq k} a_i \sigma_{i2})$. Here, $\mathcal{A}_2$ simulates the malicious KGC who knows the primary secret key sent by $\mathcal{B}$, and d_{ID^*} can be computed. Obviously, σ_{BLS} is a valid BLS signature under the public key pk_{BLS} and the implied secret key is $s_k + r_k$. Since the BLS signature has been proved secure, and then our certificateless subgroup multi-signature scheme is unforgeable against the adversary $\mathcal{A}_2$.

Alternatively, and more fundamentally, we can construct a direct reduction from the success of $\mathcal{A}_2$ to the CDH assumption. The challenger $\mathcal{B}$ embeds its CDH challenge $(\alpha G, \beta G)$ by setting the target user's public key to $pk^* = \beta G$ and programming the random oracle H_3 such that $H_3(m^*) = \alpha G$ for the target forgery message m^* . Since $\mathcal{A}_2$ possesses the primary secret key x , it can perfectly simulate all Extract queries. Following the extraction via the knowledge soundness of the argument system and algebraic manipulation similar to the case for $\mathcal{A}_1$, a valid forgery by $\mathcal{A}_2$ allows $\mathcal{B}$ to compute $\alpha \beta G$, thereby solving the CDH instance. This provides a tight reduction to the standard CDH assumption.

To sum up, our certificateless subgroup multi-signature scheme is unforgeable under the CDH assumption and the BLS signature.

G. Dynamic Group Management

For simplicity, our scheme assumes a static set of n signers with fixed identities and public keys. In practical deployments, the group may evolve over time as members join or leave. This can be supported by periodically re-initializing the system parameters (e.g., updating the set $\mathcal{ID}$ and the corresponding vector lengths) and redistributing the hash values $H_2(ID_i, pk_i, \mathbf{pk}, \mathcal{ID})$ to reflect the current group composition. Since the inner product argument and the Univariate SumCheck operate on vectors of length equal to the current group size, dynamic membership can be accommodated through such periodic updates without altering the core cryptographic primitives. Moreover, the certificateless nature simplifies key management during membership changes, as no certificate authority is required for new joiners.

H. Performance Analysis

In this section, we first compare our scheme with existing threshold signature and multi-signature schemes under realistic system parameters to highlight the advantages of avoiding the DKG protocol and achieving anonymity with logarithmic overhead. We then present implementation results to evaluate practical performance.

TABLE II
COMPARISON WITH THRESHOLD SIGNATURES

Scheme	Signature size	Verification key size	Verification cost	Aggregation key size	Aggregation key cost	Updating threshold cost	Architecture
Chelsea et al. [23]	$1\mathbb{G}$	$1\mathbb{G}$	$O(1)$	$O(n)$	$O(n \log n)$	$O(n)$	PKI
Groth et al. [21]	$3\mathbb{G}$	$7\mathbb{G}$	$O(1)$	very large	Very High	$O(n)$	PKI
Das et al. [26]	$8\mathbb{G} + 1Z_q$	$7\mathbb{G}$	$O(1)$	$O(n)$	$O(n \log n)$	$O(n)$	PKI
Our work	$6\mathbb{G} + 2Z_q$	$2\mathbb{G}$	$O(1)$	$O(n)$	$O(n \log n)$	no	certificateless

1) *Comparison With Threshold Signatures*: The subgroup multi-signature scheme is similar to common threshold signatures in that it does not require the DKG protocol. Therefore, we first compare our scheme with existing threshold signature schemes. We assume that the compared threshold signature schemes involve n signers and a threshold t , using (n, t) Shamir secret sharing [19]. Chelsea's scheme is a typical example with constant signature size and verification time [23]. Groth proposed a threshold signature based on SNARKs, where each signer generates their individual signatures using their secret key shares and sends them to the aggregator [21]. The aggregator then generates a SNARK proof to verify that it has received at least t valid signatures from the participating signers. However, not only is the cost of DKG in the setup phase significant, but the SNARK proof generation by the aggregator is also computationally expensive. Das et al. proposed a threshold signature scheme based on inner product arguments, which supports any arbitrary subgroup of signers and flexible thresholds [26]. However, the secret key still needs to be redistributed when the threshold changes. Moreover, Das et al.'s scheme remains within the PKI construction.

To provide a concrete comparison, we consider a realistic setting with 128-bit security, using a pairing-friendly curve (e.g., BLS12-381) where $|\mathbb{G}| = 48$ bytes. The number of signers is set to $n = 512$ and the threshold $t = 256$. Under these parameters, the DKG setup in existing schemes requires $O(n)$ exponentiations and multiple rounds of communication, whereas our scheme incurs no such cost. Moreover, threshold updates in prior work necessitate re-running DKG, while our scheme supports threshold changes without additional setup. Table II shows the comparison with threshold signatures. From Table II, we can see that compared with threshold signatures, the main advantage of our scheme is no DKG (no updating threshold cost), while achieving similar verification efficiency.

2) *Computational Complexity Comparison*: Several multi-signature schemes have been proposed for consensus mechanisms in blockchain, including Ma et al.'s Chameleon hash-based scheme [4], Maxwell's Schnorr-based schemes for Bitcoin [5], Kojima et al.'s Schnorr-based scheme with key aggregation [6], and Galindo et al.'s subgroup multi-signature scheme based on BLS signatures [3]. Our construction can be easily adapted to a common multi-signature scheme by removing the inner product argument and making the public keys of participating signers publicly available. We compare our scheme with the aforementioned schemes in terms of signature combining time, key aggregating time, multi-signature verification time, and communication time during verification. The communication time primarily refers to the transcript size between the prover $\mathcal{P}$

(aggregator) and the verifier $\mathcal{V}$. Additionally, the multi-signature verification time is divided into the times required by $\mathcal{P}$ and $\mathcal{V}$ in the inner product argument protocol. Let M and E denote the number of field multiplications and exponentiations (group multiplications), respectively. Let Pr denote the bilinear pairing operation, and we omit group addition and field addition. We assume the number of participating signers is n . Table III shows the comparison of multi-signature schemes, in which we especially adds Galindo et al.'s subgroup multi-signature scheme as a comparison baseline. Its communication overhead is $O(n)$ because the verifier needs the list of all participants' public keys. In Table III, we splits our scheme into "Basic" (without privacy) and "Full" versions allows for a fair assessment: the extra cost for advanced features (anonymity, threshold) is a logarithmic overhead versus a linear one.

For a concrete evaluation under realistic parameters, we set $n = 512$ and adopt the same 128-bit security assumption. Under these parameters, the communication overhead of Galindo et al. is $O(n)$ group elements, i.e., $512 \times 48 = 24$ KB, while our Full scheme requires only $O(\log n) = 9$ group elements (for the inner product argument) plus the SumCheck proof of $3|\mathbb{G}|$, totaling approximately $12 \times 48 = 576$ bytes, a reduction of more than 40 times. The verification cost in our Full scheme involves $O(\log n)$ exponentiations (around 9 exponentiations) plus a constant number of pairings, which for $n = 512$ is comparable to or even lower than the linear communication processing overhead of Galindo et al.

As shown in Table III, when compared to the subgroup multi-signature scheme by Galindo et al. [3], our scheme avoids transmitting the linear-size list of public keys during verification. Instead, it uses a logarithmic-size zero-knowledge proof (the inner product argument), significantly reducing communication bandwidth. This is an additional benefit arising from the signer anonymity property.

3) *Implementation Analysis*: Our certificateless subgroup multi-signature scheme is implemented using the Java Pairing-Based Cryptography (JPBC) Library and the Java programming language. The implementation platform is a Lenovo Legion R9000P laptop equipped with an AMD Ryzen 7 6800H 3.20 GHz CPU and 16 GB of memory. The generation of a partial signature, which combines BLS signatures and Sakai's identity-based signatures, takes approximately 86 ms. Fig. 1 illustrates the time costs of generating an aggregated public key, which increases almost linearly with the number of participating signers. When the number of participating signers is less than 500, generating an aggregated public key requires only a few seconds. To provide a more intuitive performance comparison with existing schemes, we implemented two key baselines: the

TABLE III
COMPUTATIONAL COMPLEXITY COMPARISON AMONG MULTI-SIGNATURES (SUBGROUP MULTI-SIGNATURE)

Scheme	Architecture	Subgroup Support / Anonymity	Signature Combine Time	Key Aggregation Time	Comm. Overhead (Verify)	Multi-signature Verify Time
Ma et al. [4]	PKI	No	$O(n)E$	$O(n)E$	$O(1)$	$O(1)Pr$
Maxwell et al. [5]	PKI	No	$O(n)M$	$O(n)E$	$O(1)$	$O(1)E$
Kojima et al. [6]	PKI	No	$O(n)M$	$O(n)E$	$O(1)$	$O(1)E$
Galindo et al. [3]	PKI	Yes/No	$O(n)E$	$O(n)E$	$O(n)$	$O(1)Pr$
Our scheme(Basic)	Certificateless	No	$O(n)E$	$O(n)E$	$O(1)$	$O(1)Pr$
Our scheme(Full)	Certificateless	Yes/Anonymous	$O(n)E + \text{SumCheck}$	$O(n)E$	$O(\log n)$	$O(\log n)E + \text{Sum-Check}$

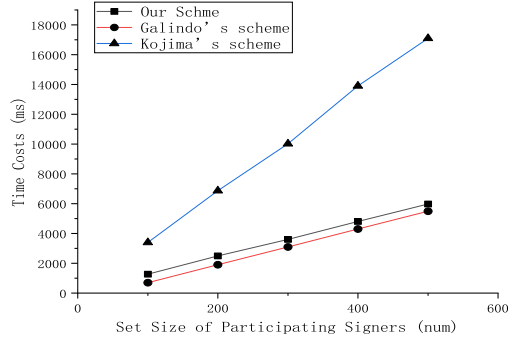


Fig. 1. Time Comparison of Key Aggregation. The x-axis represents the number of participating signers n , and the y-axis represents the time cost in milliseconds.

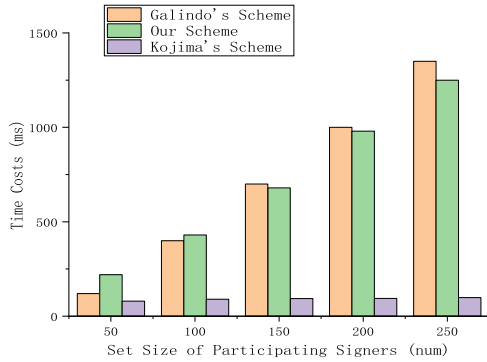


Fig. 2. Signature Verification Time Comparison. The x-axis shows the number of signers n , and the y-axis shows the verification time in milliseconds.

subgroup multi-signature scheme by Galindo et al. [3] and the highly efficient multi-signature scheme by Kojima et al. [6]. Fig. 1 compares the time cost for the key aggregation phase. As all the three schemes perform $O(n)$ scalar multiplications or point additions, their costs scale linearly with n , placing our scheme in the same order of magnitude as Galindo's. Fig. 2 shows the time costs for the verifier $\mathcal{V}$ during the Verimul phase. The verifier's time cost includes the computational overhead of the sum check. While the Galindo scheme's cryptographic verification is constant (pairing operations), the verifier must first receive and process the linear-sized ($O(n)$) list of public keys, leading to an overall linear growth in verification time. In contrast, our scheme compresses both communication and verification computation to a logarithmic scale via the inner product argument. Consequently, for a moderate to large number of participants (e.g., $n > 100$), the total verification time of our full scheme becomes lower than that of Galindo's scheme.

Compared to Kojima et al.'s scheme, which provides no subgroup or anonymity features, our scheme incurs an additional logarithmic overhead, which is the necessary and acceptable cost for enabling privacy preservation and flexible threshold validation. These experimental results strongly support the theoretical analysis in Table III, demonstrating that our scheme offers advanced functionalities while maintaining practical performance.

4) *Deployment Considerations in Consensus Environments:* In practical consensus mechanisms such as PBFT or blockchain-based protocols, the communication overhead and scalability of cryptographic primitives are critical. Our scheme's verification phase transmits only $O(\log n)$ group elements (e.g., ≈ 576 bytes for $n = 512$), which is significantly lower than the linear-size public-key lists required by prior subgroup multi-signatures [3]. This reduction directly translates to lower bandwidth consumption and faster propagation in peer-to-peer networks.

Scalability Analysis: As the number of signers grows, the aggregator's computation remains $O(n)$ (dominated by signature aggregation), while the verifier's workload is only $O(\log n)$ (inner product argument) plus $O(1)$ pairings. For $n = 1024$, the proof size increases to only $\approx 10 \log_2(1024) = 10$ group elements, maintaining logarithmic scaling. This makes the scheme suitable for large-scale consensus where verification nodes are resource-constrained.

Dynamic Membership: The certificateless design further simplifies key management by eliminating certificate authorities and reducing the overhead of key distribution in dynamic environments where participants frequently join or leave. Periodic updates to the vector length and re-computation of hash values $H_2(ID_i, pk_i, \mathbf{pk}, \mathcal{ID})$ are sufficient to accommodate membership changes without re-executing distributed key generation protocols.

IV. CONCLUSION

Multi-signature and threshold signature are two cryptographic primitives for consensus mechanisms. Unlike common multi-signature schemes, where the identities of signers are publicly known, our multi-signature scheme achieves signer anonymity, similar to threshold signatures. However, unlike threshold signatures, our scheme does not require re-executing the DKG protocol when the threshold changes. Frequent changes to the threshold can incur significant costs in DKG, especially in certificateless constructions where the cost may double. In our scheme, the privacy of participating signers is protected through the inner product argument of the aggregated public key, which keeps their identities anonymous during verification. We employ the lemma of Univariate SumCheck to compare the subgroup

size with the threshold. Any arbitrary subgroup of signers can generate a valid multi-signature, provided that the subgroup size exceeds the given threshold. Furthermore, our certificateless construction eliminates the need for public key management and addresses the key escrow problem. Thus, our certificateless subgroup multi-signature scheme with anonymity not only enhances the security of the consensus mechanism but also protects the privacy of participants, achieving a balance between security and privacy. While our scheme offers significant advantages, we acknowledge several limitations. First, the prover must collect auxiliary values from all participating signers during the signing phase, introducing coordination overhead. Second, the SumCheck proof requires bilinear pairings, which are more expensive than standard group operations. Future work may explore non-pairing-based alternatives or further optimize the inner product argument for specific deployment scenarios. Additionally, extending the scheme to support weighted thresholds (where different signers have different voting weights) remains an open problem.

REFERENCES

- [1] K. Itakura and K. Nakamura, "A public-key cryptosystem suitable for digital multisignatures," *NEC Res. Develop.*, vol. 71, pp. 1–8, 1983.
- [2] N. Z. Aitzhan and D. Svetinovic, "Security and privacy in decentralized energy trading through multi-signatures, blockchain and anonymous messaging streams," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 5, pp. 840–852, Sep./Oct. 2018.
- [3] D. Galindo and J. Liu, "Robust subgroup multi-signatures for consensus," in *Proc. Cryptographers Track RSA Conf.*, San Francisco, CA, USA, 2022, pp. 537–561.
- [4] C. B. Ma and D. K. He, "A chameleon new multi-signature based on bilinear pairing," in *Grid and Cooperative Computing*. Berlin, Germany: Springer, 2004, pp. 329–334.
- [5] G. Maxwell et al., "Simple schnorr multi-signatures with applications to bitcoin," *Designs Codes Cryptography*, vol. 87, no. 9, pp. 2139–2164, 2019.
- [6] R. Kojima et al., "A novel scheme of schnorr multi-signatures for multiple messages with key aggregation," in *Advances on Broad-Band Wireless Computing, Communication and Applications*, Berlin, Germany: Springer, 2021, pp. 284–295.
- [7] A. Shamir, "Identity-based cryptosystems and signature schemes," in *Advances in Cryptology*. Berlin, Germany: Springer, 2007, pp. 47–53.
- [8] R. Tso, Z.-Y. Liu, and Y.-F. Tseng, "Identity-based blind multisignature from lattices," *IEEE Access*, vol. 7, pp. 182916–182923, 2019.
- [9] L. Wei, Z. Cao, and X. Dong, "Secure identity-based multisignature schemes under quadratic residue assumptions," *Secure Commun. Netw.*, vol. 6, no. 6, pp. 689–701, 2013.
- [10] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Advances in Cryptology - ASIACRYPT 2003*. Berlin, Germany: Springer, 2003, pp. 452–473.
- [11] Y. Chen, M. Li, and Z. Zhang, "Stronger accountability in subgroup multi-signatures with tight reductions," in *Advances in Cryptology - ASIACRYPT 2023*. Berlin, Germany: Springer, 2023.
- [12] X. Y. Huang, W. Susilo, Y. Mu, and F. T. Zhang, "On the security of certificateless signature schemes from Asiacypt," in *Proc. Int. Conf. Cryptology Netw. Secur.*, Y. G. Desmedt, H. Wang, Y. Mu, and Y. Li, eds. Berlin, Germany: Springer, 2005, pp. 13–25.
- [13] M. H. Au, J. Chen, J. K. Liu, Y. Mu, D. S. Wong, and G. Yang, "Malicious KGC attack in certificateless cryptography," in *Proc. ACM Symp. Inf. Comput. Commun. Secur.*, F. Bao and S. Miller, eds. New York, NY, USA: ACM Press, 2007, pp. 302–311.
- [14] S. K. Langford, "Weakness in some threshold cryptosystems," in *Proc. 16th Annu. Int. Cryptology Conf. Adv. Cryptology*, 1996, pp. 74–82.
- [15] C.-M. Li, T. Hwang, and N.-Y. Lee, "Threshold-multisignature schemes where suspected forgery implies traceability of adversarial shareholders," in *Proc. Workshop Theory Appl. Cryptographic Techn.*, 1995, pp. 194–204.
- [16] S. Lu, R. Ostrovsky, A. Sahai, H. Shacham, and B. Waters, "Sequential aggregate signatures and multisignatures without random oracles," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2006, pp. 465–485.
- [17] T. Ristenpart and S. Yilek, "The power of proof-of-possession: Securing multiparty signatures against rogue-key attacks," in *Proc. 26th Annu. Int. Conf. Adv. Cryptology*, 2007, pp. 228–245.
- [18] E. Shi, "Streamlined blockchains: A simple and elegant approach (a tutorial and survey)," in *Proc. 25th Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2019, pp. 3–17.
- [19] A. Boldyreva, "A threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme," in *Public Key Cryptography PKC 2003*. Berlin, Germany: Springer, 2002, pp. 31–46.
- [20] Elrond, "Elrond: A highly scalable public blockchain via adaptive state sharding and secure proof of stake," (n.d.). [Online]. Available: <https://elrond.com/assets/files/elrond-whitepaper.pdf>
- [21] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. Annu. Int. Conf. theory Appl. Cryptographic Techn.*, Springer, 2016, pp. 305–326.
- [22] R. Bacho, J. Loss, G. Stern, and B. Wagner, "HARTS: High-threshold, adaptively secure, and robust threshold schnorr signatures," in *Advances in Cryptology C ASIACRYPT 2024*, K. M. Chung and Y. Sasaki, Eds. Singapore: Springer, 2025.
- [23] C. Komlo and I. Goldberg, "FROST: Flexible round-optimized schnorr threshold signatures," in *Selected Areas in Cryptography*, O. Dunkelman, M. J. Jacobson, and C. O'Flynn, eds. Cham, Switzerland: Springer, 2021.
- [24] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2018, pp. 315–334.
- [25] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, "Aurora: Transparent succinct arguments for R1CS," in *Proc. Annu. Int. Conf. Theory Appl. cryptographic Techn.*, Springer, 2019, pp. 103–128.
- [26] S. Das, P. Camacho, Z. Xiang, J. Nieto, B. Bnz, and L. Ren, "Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 356–370.
- [27] B. Chen, B. Benedikt, D. Boneh, and Z. Zhang, "HyperPlonk: Plonk with linear-time prover and high-degree custom gates," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2023, Art. no. 499C530.
- [28] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, "Marlin: Preprocessing zkSNARKs with universal and updatable SRS," in *EUROCRYPT 2020 (Lecture Notes in Computer Science)*, Berlin, Germany: Springer, 2020, pp. 738–768.
- [29] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," *Cryptol. ePrint Arch.*, Art. no. 2019953. [Online]. Available: <https://eprint.iacr.org/2019/953>
- [30] J. Thaler, "Time-optimal interactive proofs for circuit evaluation," in *CRYPTO (Lecture Notes in Computer Science)*, R. Canetti and J. A. Garay, Eds. Berlin, Germany: Springer, 2013, pp. 71–89.
- [31] M. Campanelli, D. Fiore, and A. Querol, "LegoSNARK: Modular design and composition of succinct zero-knowledge proofs," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 2075–2092.
- [32] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song, "Libra: Succinct zero-knowledge proofs with optimal prover computation," in *Proc. 39th Annu. Int. Cryptology Conf.*, 2019, pp. 733–764.
- [33] T. H. Yuen, M. F. Esgin, J. K. Liu, M. H. Au, and Z. Ding, "DualRing: Generic construction of ring signatures with efficient instantiations," in *Proc. Annu. Int. Cryptology Conf.*, 2021, pp. 251–281.
- [34] K. Yang, P. Sarkar, C. Weng, and X. Wang, "Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Y. Kim, J. Kim, G. Vigna, and E. Shi Eds., 2021, pp. 2986–3001.
- [35] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," *J. Cryptol.*, vol. 17, no. 4, pp. 297–319, 2004.
- [36] M. Cong, T. H. Yuen, and S. M. Yiu, "zkMatrix: Batched short proof for committed matrix multiplication, cryptography ePrint archive," Paper 2024/161, 2024. [Online]. Available: <https://eprint.iacr.org/2024/161>
- [37] R. Sakai, K. Ohgishi, and M. Kasahara, "Cryptosystems based on pairing," in *Proc. Symp. Cryptogr. Inf. Secur.*, 2000, pp. 354–368.
- [38] D. Boneh, M. Drijvers, and G. Neven, "Compact multi-signatures for smaller blockchains," in *Proc. 24th Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Brisbane, QLD, Australia, Dec. 2018, pp. 435–464.
- [39] J. T. Schwartz, "Fast probabilistic algorithms for verification of polynomial identities," *J. ACM*, vol. 27, no. 4, pp. 701–717, 1980.



clouding computing, and fog/edge computing.

Zhiwei Wang received the PhD degree in cryptography from the Beijing University of Posts and Telecommunications, Beijing, China, in 2009. Since 2009, he has been a professor with the School of Computer, Nanjing University of Posts and Telecommunications, Nanjing, China. He was a research associate with the University of Hong Kong from March 2014 to March 2015. He has published more than 50 journal articles and referred conference papers. His research interests include applied cryptography, security and privacy in mobile and wireless systems,



Teacher Award of the Faculty of Engineering twice (2005 and 2009). Before he joined the Department as a faculty member, he has worked as an Analyst Programmer for a couple of years. Besides basic research, he has been involving in various industrial projects.

Siuming Yiu received the bachelor degree in computer science from the Chinese University of Hong Kong, the master degree in computer and information science from Temple University, and the PhD degree in computer science from the University of Hong Kong. He received two research output prizes, one from the department in 2013 and one from the faculty in 2006. He was selected for Outstanding Teaching Award by the University in 2009, the Teaching Excellence Award in the Department in 2001, 2003, 2004, 2005, 2007, 2009, and 2010. He also received the Best